

Name: Monisha Raghini Chelle

Roll No: 2520030537

Branch: B.Tech CSE

Subject: Linux System Programming Lab

OSSP_Practical11&12

OSSP (Operating Systems And Systems Programming) - 25CS2104E

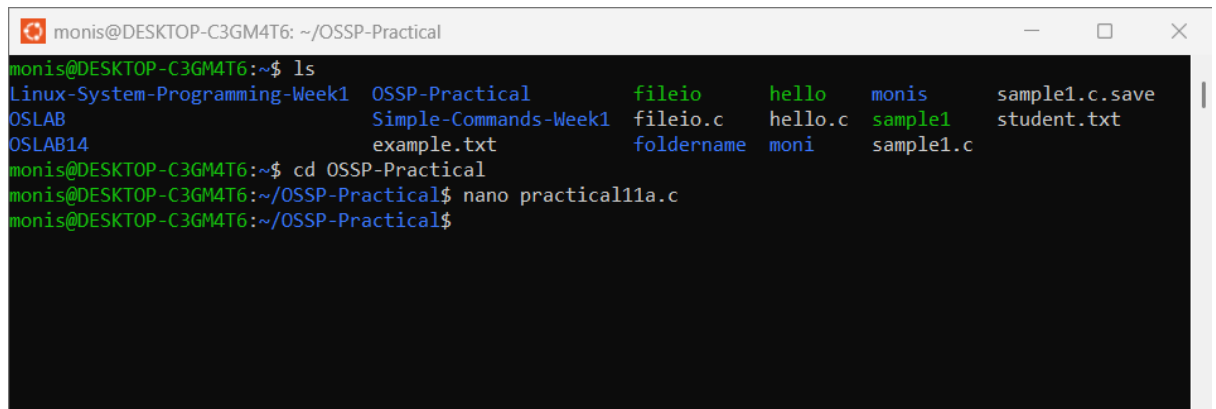
11. Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Step 1 — Create practical11a.c

cd ~/OSSP-Practical

nano practical11a.c

A terminal window titled 'monis@DESKTOP-C3GM4T6: ~/OSSP-Practical' with standard window controls. The terminal shows the following commands and output:

```
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSSP-Practical  fileio  hello  monis  sample1.c.save
OSLAB                          Simple-Commands-Week1  fileio.c  hello.c  sample1  student.txt
OSLAB14                        example.txt      foldername  moni    sample1.c

monis@DESKTOP-C3GM4T6:~$ cd OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical11a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical11a.c *
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERATIONS 1000000

long long counter = 0;

void *increment(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++) {
        counter++;
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[THREADS];

    printf("PRACTICAL 11A: RACE CONDITION\n");

    for (int i = 0; i < THREADS; i++) {
        if (pthread_create(&threads[i], NULL,
                          increment, NULL) != 0) {
            fprintf(stderr, "Thread creation failed.\n");
            return 1;
        }
    }

    for (int i = 0; i < THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter: %d\n", THREADS * ITERATIONS);
    printf("Actual counter : %lld\n", counter);

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

Save using Ctrl + O, Enter, then Ctrl + X.

```
Write to File: practical11a.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel     M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Step 2 — Compile and run

```
gcc -O0 -Wall -Wextra -pthread practical11a.c -o practical11a
./practical11a
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical11a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -O0 -Wall -Wextra -pthread practical11a.c -o practical11a
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical11a
PRACTICAL 11A: RACE CONDITION
Expected counter: 4000000
Actual counter : 1351597
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 3 — Create practical11b.c

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical11b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical11b.c *
#include <stdio.h>
#include <pthread.h>
#include <time.h>

#define THREADS 4
#define ITERATIONS 1000000

long long counter = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[THREADS];

    struct timespec start, end;

    printf("PRACTICAL 11B: MUTEX SYNCHRONIZATION\n");

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < THREADS; i++) {
        if (pthread_create(&threads[i], NULL,
                          increment, NULL) != 0) {
            fprintf(stderr, "Thread creation failed.\n");
            return 1;
        }
    }

    for (int i = 0; i < THREADS; i++) {
```

Help	Write Out	Where Is	Cut	Execute	Location	Undo
Exit	Read File	Replace	Paste	Justify	Go To Line	Redo



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical11b.c *
    counter++;

    pthread_mutex_unlock(&lock);
}

return NULL;

int main(void)

    pthread_t threads[THREADS];

    struct timespec start, end;

    printf("PRACTICAL 11B: MUTEX SYNCHRONIZATION\n");

    clock_gettime(CLOCK_MONOTONIC, &start);

    for (int i = 0; i < THREADS; i++) {
        if (pthread_create(&threads[i], NULL,
                          increment, NULL) != 0) {
            fprintf(stderr, "Thread creation failed.\n");
            return 1;
        }
    }

    for (int i = 0; i < THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    clock_gettime(CLOCK_MONOTONIC, &end);

    double seconds =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    printf("Expected counter: %d\n", THREADS * ITERATIONS);
    printf("Actual counter  : %lld\n", counter);
    printf("Execution time : %.6f seconds\n", seconds);

    pthread_mutex_destroy(&lock);

    return 0;

G Help      ^O Write Out ^F Where Is ^K Cut      ^T Execute  ^C Location  M-U Undo
X Exit      ^R Read File ^\ Replace  ^U Paste    ^_ Justify  ^/ Go To Line M-F Redo

Write to File: practical11b.c
^G Help      M-D DOS Format  M-A Append    M-B Backup File ^T Browse
^C Cancel     M-M Mac Format   M-P Prepend   ^O Discard buffer
```

Step 4 — Compile and run

```
gcc -O0 -Wall -Wextra -pthread practical11b.c -o practical11b
./practical11b
```

```
Select monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical11b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -O0 -Wall -Wextra -pthread practical11b.c -o practical11b
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical11b
PRACTICAL 11B: MUTEX SYNCHRONIZATION
Expected counter: 4000000
Actual counter : 4000000
Execution time : 0.222729 seconds
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Final verification

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
ls -l practical11a.c practical11b.c
-rw-r--r-- 1 monis monis 791 Sep 28 21:26 practical11a.c
-rw-r--r-- 1 monis monis 1258 Sep 28 21:29 practical11b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical11a
PRACTICAL 11A: RACE CONDITION
Expected counter: 4000000
Actual counter : 1729481
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical11b
PRACTICAL 11B: MUTEX SYNCHRONIZATION
Expected counter: 4000000
Actual counter : 4000000
Execution time : 0.216909 seconds
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 5 — Compare the results

Feature	Without mutex	With mutex
Shared counter	Updated concurrently	Updated inside a critical section
Result	May be incorrect or inconsistent	Correct and consistent
Race condition	Present	Prevented
Execution time	Often lower, but result is unreliable	May be higher due to locking overhead
Synchronization	None	pthread_mutex_lock() and pthread_mutex_unlock()

12. Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

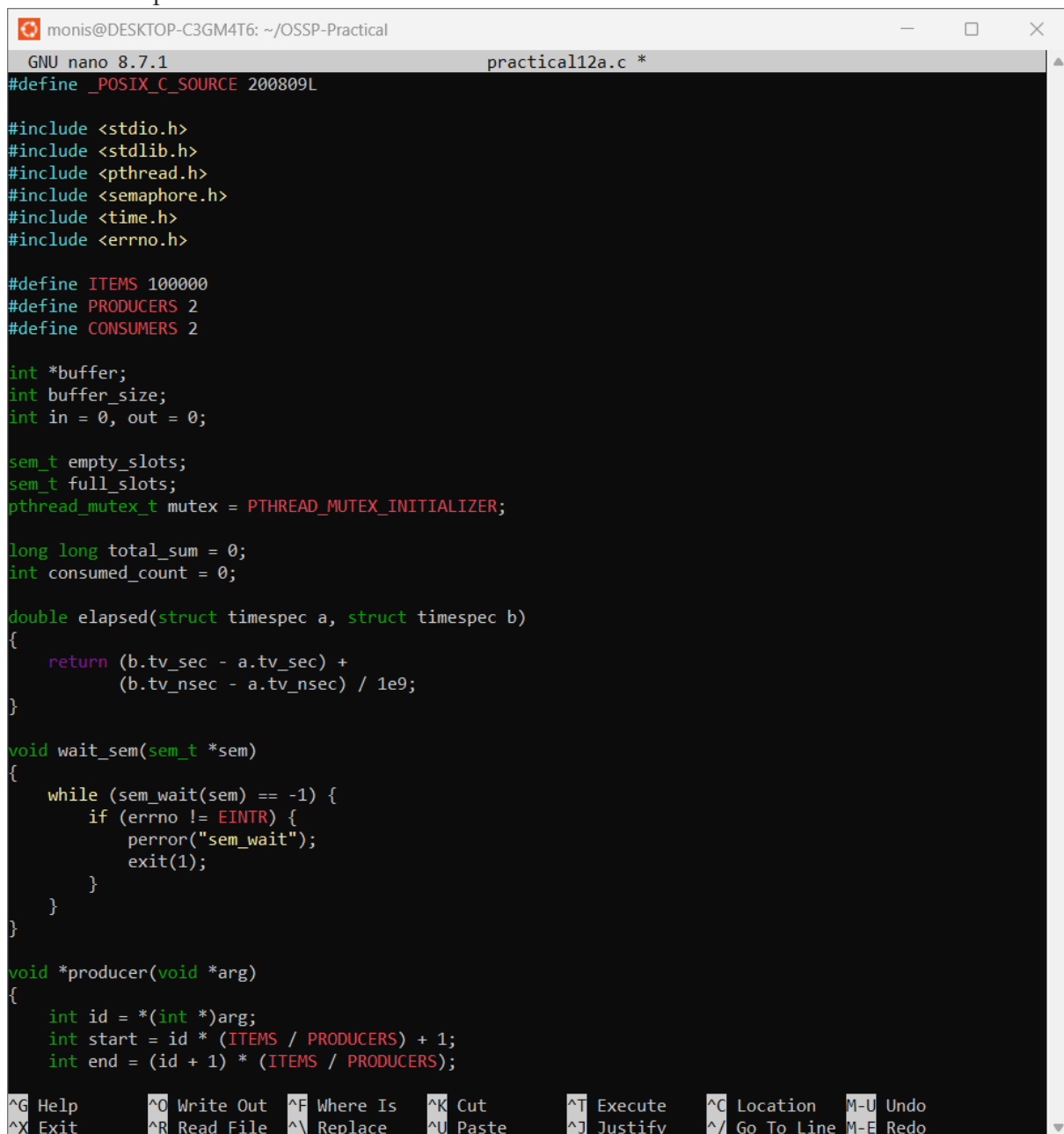
Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Step 1 — Create practical12a.c

In Ubuntu:

cd ~/OSSP-Practical

nano practical12a.c



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical12a.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>
#include <errno.h>

#define ITEMS 100000
#define PRODUCERS 2
#define CONSUMERS 2

int *buffer;
int buffer_size;
int in = 0, out = 0;

sem_t empty_slots;
sem_t full_slots;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

long long total_sum = 0;
int consumed_count = 0;

double elapsed(struct timespec a, struct timespec b)
{
    return (b.tv_sec - a.tv_sec) +
        (b.tv_nsec - a.tv_nsec) / 1e9;
}

void wait_sem(sem_t *sem)
{
    while (sem_wait(sem) == -1) {
        if (errno != EINTR) {
            perror("sem_wait");
            exit(1);
        }
    }
}

void *producer(void *arg)
{
    int id = *(int *)arg;
    int start = id * (ITEMS / PRODUCERS) + 1;
    int end = (id + 1) * (ITEMS / PRODUCERS);
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical12a.c *
int end = (id + 1) * (ITEMS / PRODUCERS);

for (int value = start; value <= end; value++) {
    wait_sem(&empty_slots);

    pthread_mutex_lock(&mutex);

    buffer[in] = value;
    in = (in + 1) % buffer_size;

    pthread_mutex_unlock(&mutex);

    sem_post(&full_slots);
}

return NULL;

}

void *consumer(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITEMS / CONSUMERS; i++) {
        wait_sem(&full_slots);

        pthread_mutex_lock(&mutex);

        int value = buffer[out];
        out = (out + 1) % buffer_size;

        total_sum += value;
        consumed_count++;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty_slots);
    }

    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "Usage: %s BUFFER_SIZE\n", argv[0]);
    }
}
```

⌘ Help ⌘ Write Out ⌘ Where Is ⌘ Cut ⌘ Execute ⌘ Location M-U Undo
⌘ Exit ⌘ Read File ⌘ Replace ⌘ Paste ⌘ Justify ⌘ Go To Line M-E Redo


```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical12a.c *
if (argc != 2) {
    fprintf(stderr, "Usage: %s BUFFER_SIZE\n", argv[0]);
    return 1;
}

buffer_size = atoi(argv[1]);

if (buffer_size <= 0 || buffer_size > 100000) {
    fprintf(stderr, "Buffer size must be 1 to 100000.\n");
    return 1;
}

buffer = malloc((size_t)buffer_size * sizeof(int));

if (buffer == NULL) {
    perror("malloc");
    return 1;
}

if (sem_init(&empty_slots, 0, (unsigned int)buffer_size) == -1 ||
    sem_init(&full_slots, 0, 0) == -1) {
    perror("sem_init");
    free(buffer);
    return 1;
}

pthread_t producers[PRODUCERS];
pthread_t consumers[CONSUMERS];
int ids[PRODUCERS];

struct timespec start, end;
clock_gettime(CLOCK_MONOTONIC, &start);

for (int i = 0; i < CONSUMERS; i++)
    pthread_create(&consumers[i], NULL, consumer, NULL);

for (int i = 0; i < PRODUCERS; i++) {
    ids[i] = i;
    pthread_create(&producers[i], NULL, producer, &ids[i]);
}

for (int i = 0; i < PRODUCERS; i++)
    pthread_join(producers[i], NULL);

for (int i = 0; i < CONSUMERS; i++)

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical12a.c *
clock_gettime(CLOCK_MONOTONIC, &start);

for (int i = 0; i < CONSUMERS; i++)
    pthread_create(&consumers[i], NULL, consumer, NULL);

for (int i = 0; i < PRODUCERS; i++) {
    ids[i] = i;
    pthread_create(&producers[i], NULL, producer, &ids[i]);
}

for (int i = 0; i < PRODUCERS; i++)
    pthread_join(producers[i], NULL);

for (int i = 0; i < CONSUMERS; i++)
    pthread_join(consumers[i], NULL);

clock_gettime(CLOCK_MONOTONIC, &end);

double seconds = elapsed(start, end);
long long expected_sum =
    (long long)ITEMS * (ITEMS + 1) / 2;

printf("PRACTICAL 12A - PRODUCER CONSUMER\n");
printf("Buffer size: %d\n", buffer_size);
printf("Produced: %d\n", ITEMS);
printf("Consumed: %d\n", consumed_count);
printf("Expected sum: %lld\n", expected_sum);
printf("Actual sum: %lld\n", total_sum);
printf("Execution time: %.6f seconds\n", seconds);
printf("Throughput: %.2f items/second\n",
    seconds > 0 ? ITEMS / seconds : 0.0);

if (consumed_count == ITEMS && total_sum == expected_sum)
    printf("Synchronization: PASS\n");
else
    printf("Synchronization: FAIL\n");

sem_destroy(&empty_slots);
sem_destroy(&full_slots);
pthread_mutex_destroy(&mutex);
free(buffer);

return 0;
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^V Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Save using Ctrl + O, Enter, then Ctrl + X.

```
Write to File: practical12a.c
^G Help M-D DOS Format M-A Append M-B Backup File ^T Browse
^C Cancel M-M Mac Format M-P Prepend ^O Discard buffer
```

Step 2 — Compile and run with different buffer sizes

```
gcc -Wall -Wextra -O2 -pthread practical12a.c -o practical12a
```

```
./practical12a 1
```

```
./practical12a 10
```

```
./practical12a 100
```

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical12a.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra -O2 -pthread practical12a.c -o practical12a
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical12a 1
PRACTICAL 12A - PRODUCER CONSUMER
Buffer size: 1
Produced: 100000
Consumed: 100000
Expected sum: 5000050000
Actual sum: 5000050000
Execution time: 4.807281 seconds
Throughput: 20801.78 items/second
Synchronization: PASS
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical12a 10
PRACTICAL 12A - PRODUCER CONSUMER
Buffer size: 10
Produced: 100000
Consumed: 100000
Expected sum: 5000050000
Actual sum: 5000050000
Execution time: 0.457761 seconds
Throughput: 218454.75 items/second
Synchronization: PASS
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical12a 100
PRACTICAL 12A - PRODUCER CONSUMER
Buffer size: 100
Produced: 100000
Consumed: 100000
Expected sum: 5000050000
Actual sum: 5000050000
Execution time: 0.038599 seconds
Throughput: 2590735.59 items/second
Synchronization: PASS
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 4 — Create practical12b.c

nano practical12b.c

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical12b.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock_a = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock_b = PTHREAD_MUTEX_INITIALIZER;

pthread_barrier_t barrier;
int prevention = 0;

void *thread_one(void *arg)
{
    (void)arg;

    pthread_mutex_lock(&lock_a);
    printf("Thread 1 acquired Resource A\n");
    fflush(stdout);

    if (!prevention)
        pthread_barrier_wait(&barrier);

    pthread_mutex_lock(&lock_b);
    printf("Thread 1 acquired Resource B\n");

    pthread_mutex_unlock(&lock_b);
    pthread_mutex_unlock(&lock_a);

    printf("Thread 1 completed\n");
    return NULL;
}

void *thread_two(void *arg)
{
    (void)arg;

    if (prevention) {
        /* Both threads acquire resources in the same order. */
        pthread_mutex_lock(&lock_a);
        printf("Thread 2 acquired Resource A\n");

        pthread_mutex_lock(&lock_b);
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^/ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical12b.c *
    pthread_mutex_lock(&lock_b);
    printf("Thread 2 acquired Resource B\n");

    pthread_mutex_unlock(&lock_b);
    pthread_mutex_unlock(&lock_a);

    printf("Thread 2 completed\n");
} else {
    /* Opposite order creates a deadlock. */
    pthread_mutex_lock(&lock_b);
    printf("Thread 2 acquired Resource B\n");
    fflush(stdout);

    pthread_barrier_wait(&barrier);

    pthread_mutex_lock(&lock_a);
    printf("Thread 2 acquired Resource A\n");

    pthread_mutex_unlock(&lock_a);
    pthread_mutex_unlock(&lock_b);
}

return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2 ||
        (strcmp(argv[1], "deadlock") != 0 &&
         strcmp(argv[1], "prevent") != 0)) {
        fprintf(stderr,
            "Usage: %s deadlock|prevent\n", argv[0]);
        return 1;
    }

    prevention = strcmp(argv[1], "prevent") == 0;

    pthread_t t1, t2;

    if (!prevention)
        pthread_barrier_init(&barrier, NULL, 2);

    printf("PRACTICAL 12B - %s\n",
        prevention ? "DEADLOCK PREVENTION" :
        "DEADLOCK DEMONSTRATION");

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
GNU nano 8.7.1 practical12b.c *
    pthread_mutex_unlock(&lock_b);
}

return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2 ||
        (strcmp(argv[1], "deadlock") != 0 &&
         strcmp(argv[1], "prevent") != 0)) {
        fprintf(stderr,
            "Usage: %s deadlock|prevent\n", argv[0]);
        return 1;
    }

    prevention = strcmp(argv[1], "prevent") == 0;

    pthread_t t1, t2;

    if (!prevention)
        pthread_barrier_init(&barrier, NULL, 2);

    printf("PRACTICAL 12B - %s\n",
        prevention ? "DEADLOCK PREVENTION" :
            "DEADLOCK DEMONSTRATION");
    fflush(stdout);

    pthread_create(&t1, NULL, thread_one, NULL);
    pthread_create(&t2, NULL, thread_two, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    if (!prevention)
        pthread_barrier_destroy(&barrier);

    pthread_mutex_destroy(&lock_a);
    pthread_mutex_destroy(&lock_b);

    printf("Both threads completed successfully.\n");

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo

}
Write to File: practical12b.c
^G Help      M-D DOS Format M-A Append   M-B Backup File ^T Browse
^C Cancel     M-M Mac Format M-P Prepend  ^O Discard buffer
```

Step 5 — Compile and demonstrate deadlock

gcc -Wall -Wextra -pthread practical12b.c -o practical12b

timeout 3s ./practical12b deadlock

echo "Exit status: \$?"

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ nano practical12b.c
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ gcc -Wall -Wextra -pthread practical12b.c -o practical12b
timeout 3s ./practical12b deadlock
echo "Exit status: $?"
PRACTICAL 12B - DEADLOCK DEMONSTRATION
Thread 1 acquired Resource A
Thread 2 acquired Resource B
Exit status: 124
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 6 — Demonstrate deadlock prevention

`./practical12b prevent`

```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ./practical12b prevent
PRACTICAL 12B - DEADLOCK PREVENTION
Thread 1 acquired Resource A
Thread 1 acquired Resource B
Thread 1 completed
Thread 2 acquired Resource A
Thread 2 acquired Resource B
Thread 2 completed
Both threads completed successfully.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```

Step 7 — Record the four necessary conditions for deadlock

Condition	Explanation
Mutual exclusion	Only one thread can hold each mutex at a time.
Hold and wait	Each thread holds one mutex while waiting for the other.
No preemption	A mutex cannot normally be forcibly taken from its owner.
Circular wait	Thread 1 waits for B held by Thread 2, while Thread 2 waits for A held by Thread 1.

Step 8 — Final verification

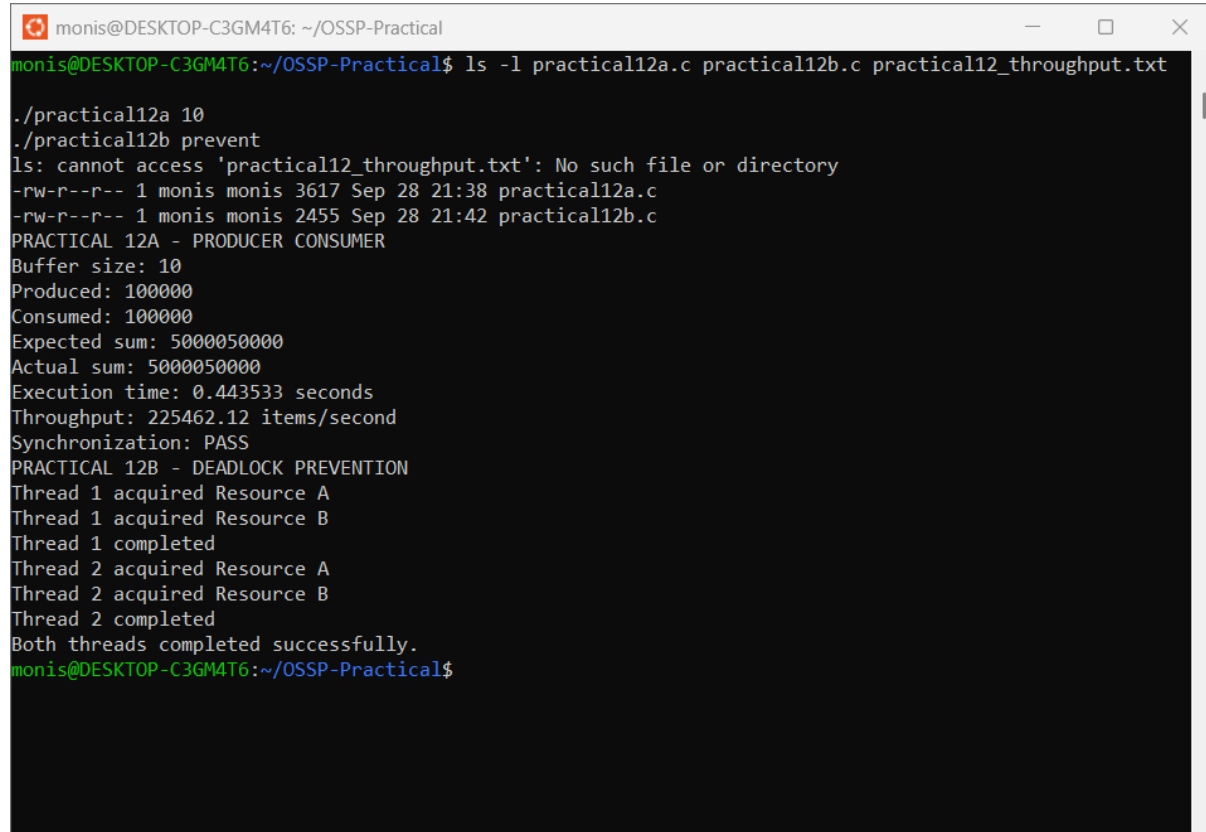
Run:

`cd ~/OSSP-Practical`

```
ls -l practical12a.c practical12b.c practical12_throughput.txt
```

```
./practical12a 10
```

```
./practical12b prevent
```



```
monis@DESKTOP-C3GM4T6: ~/OSSP-Practical
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$ ls -l practical12a.c practical12b.c practical12_throughput.txt
ls: cannot access 'practical12_throughput.txt': No such file or directory
-rw-r--r-- 1 monis monis 3617 Sep 28 21:38 practical12a.c
-rw-r--r-- 1 monis monis 2455 Sep 28 21:42 practical12b.c
PRACTICAL 12A - PRODUCER CONSUMER
Buffer size: 10
Produced: 100000
Consumed: 100000
Expected sum: 5000050000
Actual sum: 5000050000
Execution time: 0.443533 seconds
Throughput: 225462.12 items/second
Synchronization: PASS
PRACTICAL 12B - DEADLOCK PREVENTION
Thread 1 acquired Resource A
Thread 1 acquired Resource B
Thread 1 completed
Thread 2 acquired Resource A
Thread 2 acquired Resource B
Thread 2 completed
Both threads completed successfully.
monis@DESKTOP-C3GM4T6:~/OSSP-Practical$
```